

Preparar la Plataforma Tecnológica

Documentación del proyecto Moda Mágica · Despliegue en Render

Preparar la plataforma tecnológica es la etapa en la que se define y configura la infraestructura sobre la cual se ejecutará el sistema en un entorno de producción accesible públicamente. Para el proyecto **Moda Mágica** —una plataforma de comercio electrónico de moda construida con React/Vite en el frontend, Node.js/Express en el backend y PostgreSQL como motor de base de datos— esta etapa consistió en seleccionar un proveedor de hosting, aprovisionar cada uno de los servicios requeridos, vincularlos al repositorio de control de versiones y resolver los ajustes de configuración necesarios para que los tres componentes se comunicaran correctamente entre sí en un entorno distinto al de desarrollo local.

El proveedor seleccionado fue **Render**, una plataforma de hosting en la nube que permite desplegar bases de datos administradas, servicios web (APIs) y sitios estáticos de manera integrada, con despliegue continuo directamente desde GitHub. A continuación se describe el proceso completo: la arquitectura resultante, la configuración de cada servicio, las variables de entorno necesarias, los problemas encontrados durante el despliegue y su solución.

1. Arquitectura general

La plataforma quedó organizada en tres servicios independientes que se comunican entre sí mediante peticiones HTTP y variables de entorno, en lugar de depender de rutas o direcciones fijas como ocurre en un entorno local (*localhost*):

- **Base de datos:** instancia PostgreSQL administrada por Render, donde reside el esquema completo de 18 tablas del sistema (usuarios, productos, categorías, pedidos, descuentos, colores, tallas, imágenes, entre otras).
- **Backend:** servicio web Node.js/Express que expone la API REST, gestiona la lógica de negocio, la autenticación, el procesamiento de pagos y la conexión a la base de datos.
- **Frontend:** sitio estático generado con React/Vite, que consume la API del backend a través de la variable de entorno VITE_API_URL.

Los tres servicios están vinculados al mismo repositorio de GitHub (**SantiagoGiraldoRodriguez/ModaMagica**), rama **main**, con despliegue automático: cada vez que se envía un nuevo commit a esa rama, Render reconstruye y publica automáticamente el servicio correspondiente, sin intervención manual.

2. Servicios aprovisionados

Se crearon y configuraron tres servicios bajo un mismo proyecto de Render:

Servicio	Tipo	Tecnología	Estado
ModaMagica	PostgreSQL (Free)	Base de datos relacional	Available

modamagica-backend	Web Service	Node.js / Express	Deployed
modamagica-frontend	Static Site	React / Vite	Deployed

Los tres servicios están vinculados al repositorio de GitHub **SantiagoGiraldoRodriguez/ModaMagica**, rama **main**, con despliegue automático (auto-deploy) cada vez que se hace push de un nuevo commit.

3. Variables de entorno

Uno de los ajustes centrales para que la aplicación funcionara fuera del entorno local fue eliminar todas las referencias fijas a `localhost:3000` y reemplazarlas por variables de entorno, configuradas de forma independiente en cada servicio de Render. En total se actualizaron más de 8 componentes del frontend que dependían de una URL fija.

Variable	Servicio	Propósito
VITE_API_URL	Frontend	URL base de la API del backend, consumida por todas las peticiones del cliente.
FRONTEND_URL	Backend	URL pública del frontend, usada para configurar CORS y redirecciones (p. ej. pagos).
BACKEND_URL	Backend	URL pública propia del servicio, usada en construcción de enlaces y webhooks.

Adicionalmente, el backend requirió configurar el origen permitido de **CORS** (Cross-Origin Resource Sharing) para aceptar peticiones provenientes del dominio público del frontend, así como de los orígenes de desarrollo local utilizados durante las pruebas.

4. Problemas encontrados y solución

Durante la puesta en marcha de la plataforma se presentaron varios inconvenientes propios de migrar un proyecto desde un entorno de desarrollo local a un entorno de nube. Los más relevantes fueron:

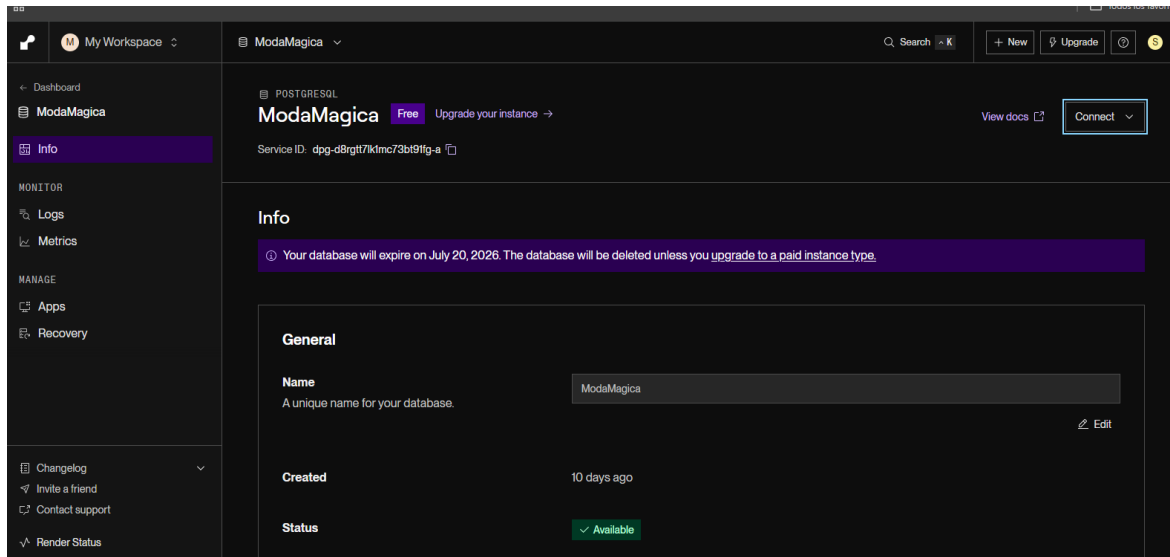
- **Carpeta node_modules versionada en Git:** al estar incluida en el repositorio, generaba errores de permisos durante el build en el entorno Linux de Render. Se solucionó agregándola al `.gitignore` y limpiando el historial de Git.
- **Interferencia de OneDrive con los archivos internos de Git:** la sincronización automática de OneDrive bloqueaba o modificaba archivos dentro de `.git`, provocando fallos intermitentes; se resolvió excluyendo la carpeta del repositorio de la sincronización.
- **Configuración incorrecta de CORS:** las primeras peticiones del frontend desplegado eran bloqueadas por el backend al no reconocer el nuevo dominio público; se ajustó la lista de orígenes permitidos para incluir todas las URLs activas.
- **Variables de entorno faltantes:** el build inicial fallaba o apuntaba a `localhost` por no tener configuradas `VITE_API_URL`, `FRONTEND_URL` y `BACKEND_URL` en el panel de Render; se definieron explícitamente en cada servicio.

- **Envío de correos bloqueado en el plan gratuito:** el envío de notificaciones por correo mediante nodemailer no funcionaba en el plan Free de Render por restricciones de puertos SMTP salientes; se migró el envío de correos de confirmación de compra al servicio **Resend**, compatible con el plan gratuito.

5. Evidencia de configuración en Render

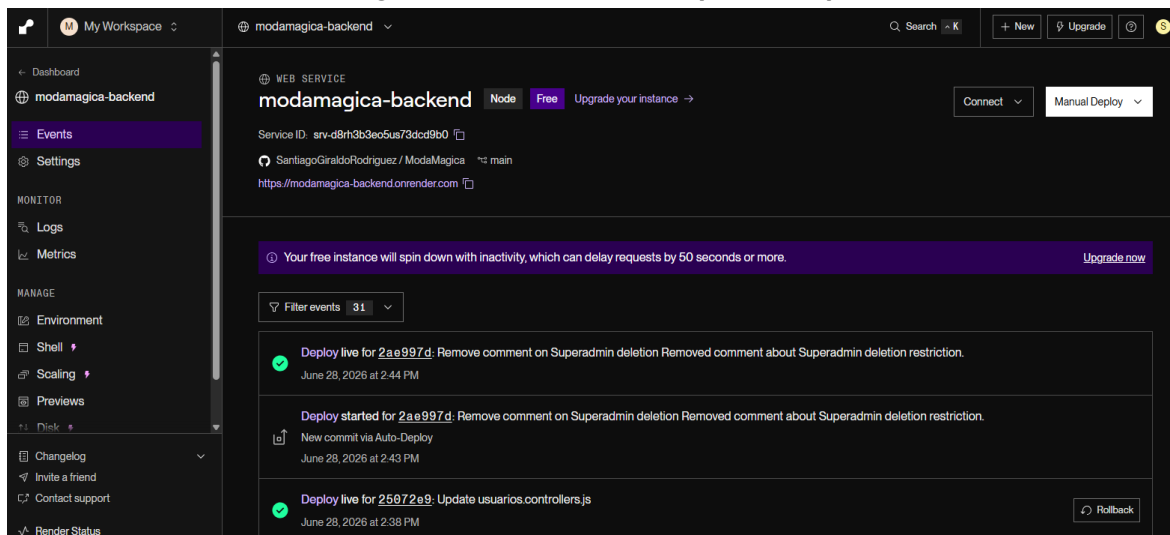
A continuación se documenta, mediante capturas de pantalla del panel de control de Render, el estado final de cada servicio una vez completado el proceso de aprovisionamiento y despliegue descrito en las secciones anteriores.

Figura 1. Base de datos PostgreSQL



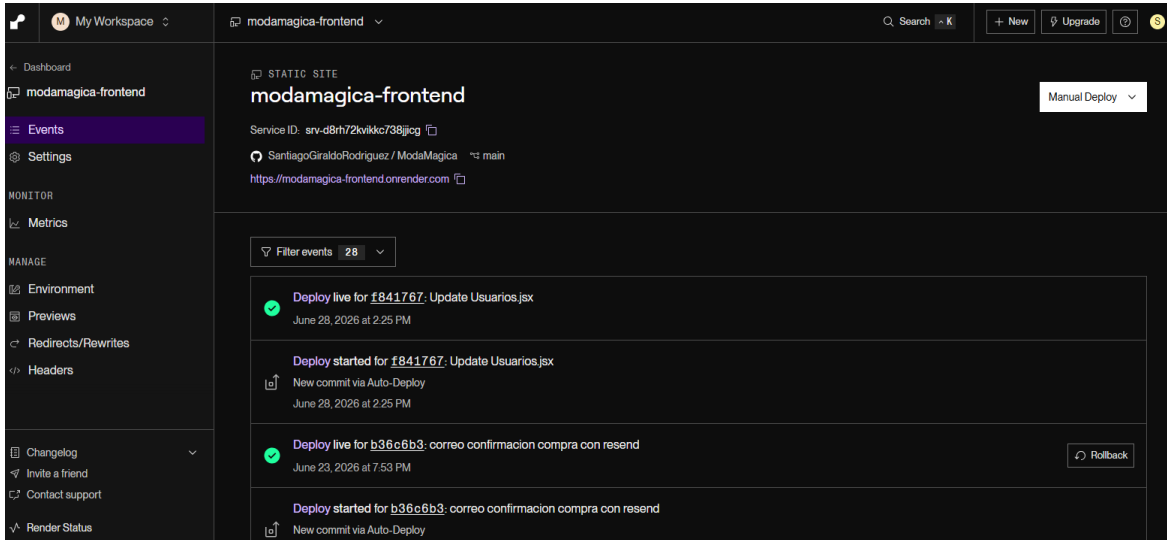
Instancia PostgreSQL «ModaMagica» en plan Free, con estado «Available» y Service ID `dpg-d8rgtt7lk1mc73bt91fg-a`. Creada como almacenamiento principal del sistema.

Figura 2. Servicio backend (API REST)



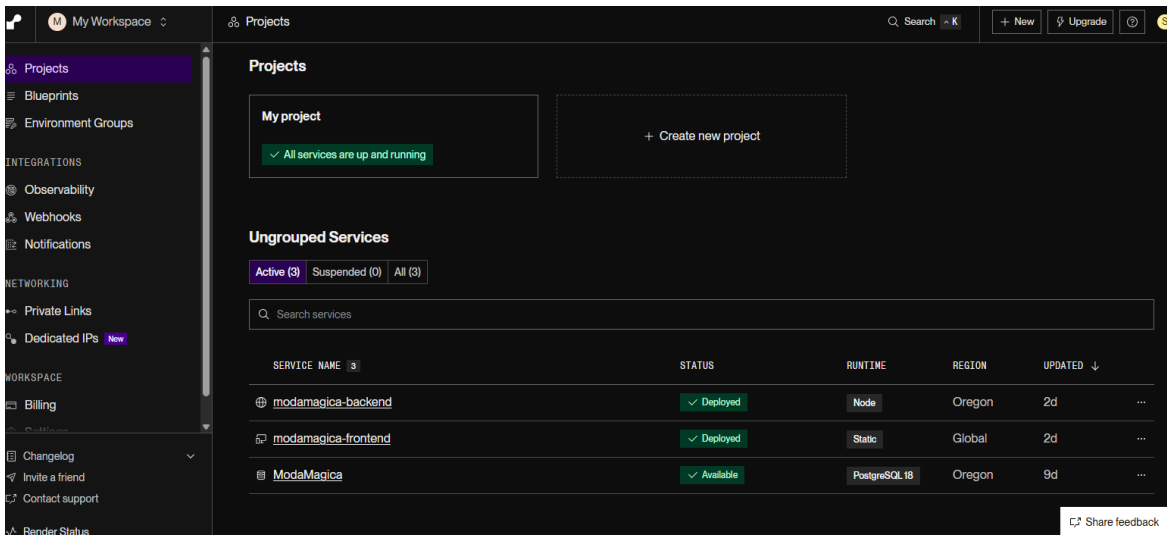
Web Service «modamagica-backend» (Node.js), conectado al repositorio ModaMagica, rama main, desplegado en <https://modamagica-backend.onrender.com> con auto-deploy activo.

Figura 3. Servicio frontend (sitio estático)



Static Site «modamagica-frontend» (React/Vite), publicado en <https://modamagica-frontend.onrender.com>, con historial de despliegues automáticos vinculados a cada commit.

Figura 4. Resumen del proyecto en Render



Vista general del proyecto con los tres servicios activos: base de datos, backend y frontend, todos en estado «Deployed» / «Available».

6. Consideraciones del plan gratuito

Render ofrece un plan gratuito por servicio, suficiente para el desarrollo y las pruebas del proyecto, pero con algunas limitaciones que es importante tener en cuenta de cara a un eventual paso a producción real:

- La base de datos PostgreSQL en plan Free tiene una fecha de expiración; si no se actualiza a un plan pago antes de esa fecha, la instancia y sus datos se eliminan.
- El servicio backend en plan Free entra en reposo (*spin down*) tras un periodo de inactividad, lo

que puede generar una demora de hasta 50 segundos o más en la primera solicitud recibida después de ese reposo.

- El sitio estático del frontend no presenta este comportamiento, ya que se sirve mediante CDN y no depende de un proceso en ejecución constante.
- Las restricciones de salida SMTP del plan gratuito impiden el envío de correos por métodos tradicionales como nodemailer, por lo que se optó por un proveedor de correo transaccional compatible (Resend).

En conjunto, esta preparación de la plataforma tecnológica permitió contar con un entorno de producción funcional, accesible públicamente, con despliegue continuo desde GitHub y comunicación correctamente configurada entre los tres servicios que componen el sistema Moda Mágica.